



Data Engineering & Data Governance: Final Project

Supervisor: Mohsen Pourpouneh

EBC2199 – Tutorial 2, Group 2

Petar Fabinger	i6347565
André Machado Paulo	i6362552
Konstanty Wilczyński	6305358
Mirza Kiliç	i6361094

Table of Contents:

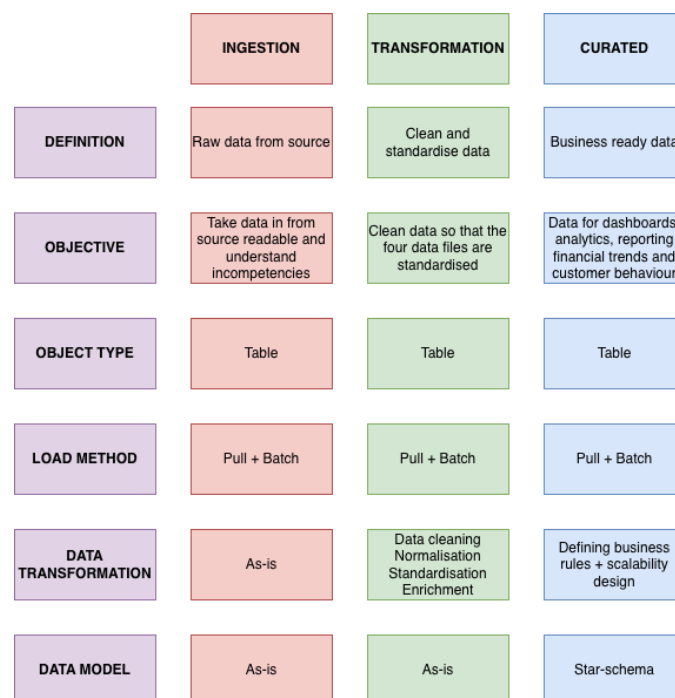
1 Project Initialisation	2
1.1 - Understanding the source Files	3
2 Ingestion	6
3 Transformation	6
4 Curated	11
5 Marts	14
5.1 mart_finance	14
5.2 mart_customer_analytics	14
5.3 mart_merchant_partnerships	15
5.4 Fraud	16
5.5 Leadership	16
6 Conclusion	17

1 Project Initialisation

The project was initialised by studying the project description and creating the github repository:

https://github.com/pfabin/clearspend_dwh/

Following this we built the data warehouse layers:



(Figure I: Data Warehouse Layers)

Most of the planning is trivial to the ETL, like taking in source data during the ingestion phase. However, some important remarks are present. We chose the **pull + batch** load method as our analysis is in OLAP nature, meaning we pull the data in bulk from the source. Also we have chosen the **kimball** star-schema design, as we want specific view-marts once the data warehouse is designed, fast queries, and it is a small analytics project, not the scale of Inmon's design.

The naming convention we chose is "snake_case", all of this information is present in the README.md on github.

1.1 - Understanding the source Files

In order to complete the transformation layer well, we investigated the individual datasets anomalies, of which we found:

transactions_data.csv:

- id
 - has no nulls or duplicates, it is **safe**
 - We assume if id has no duplicates, we can't remove rows simply because they look the same
- date
 - has no anomalies, nothing past today or nulls or too low, **safe**
- **amount**
 - currency is in the rows
 - **We assume Negative is refund, and everything USD**
- use_chip
 - **safe**
- Other id columns ("..._id")
 - **safe**
- **merchant_city**
 - "ONLINE" values should be its own column **is_online** boolean
 - After transforming the above, NULLs become NA
- **merchant_state**
 - null's must be NA
 - Also has countries mixed with US states, and no US country, so new column **merchant_country** must be made
 - 52 US states (50 + DC and Armed Forces AA)
- **zip**
 - NULL's must be NA
 - ZIP seems to be only for US states
 - ZIP is decimal and should be VARCHAR, assuming all correct and US
- **mcc**
 - codes not matching with mcc.csv
- **error**
 - column uses several different levels. Also many multiple levels in one.
 - **is_error_tech + is_error_client + error_text** have to be made for more atomicity and differentiation between our end and client end, assuming we only have the technical error, the rest are client-side

mcc.csv:

- **code**
 - is not numeric, has quotations, spaces, and duplicates
- **description**
 - Odd casing, will be standardised to title format, assuming no abbreviations like ATM present
- **notes, updated_by**
 - Both removed as they have no value to analytics, also too many missing values

users_data.csv:

- **id**
 - no nulls, but has several duplicates which must be removed
- **current_age**
 - no nulls, but does not always match birth_year with 60 rows inconsistent. This column will be dropped entirely as it will need constant updating, better keep the date only
- **retirement_age**
 - no nulls and numeric, but no information as to what this column means, also some users are retired in employment status before their retirement age value. This column will be dropped due to uncertain information for analytics.
- birth_year
 - no nulls or dates before or beyond today, **safe**
- birth_month
 - All good, values are between 1 and 12, **safe**
- gender
 - Only Male and Female, **safe**
- address
 - No odd behaviour, **safe**
- latitude + longitude
 - numeric and plausible, **safe**
- **per_capita_income**
 - stored as text, not numeric, has \$, commas, and k values.
 - Due to no atomicity with this column, we remove it, since per capita income is not specifically for this single user, but a location. *Also*, could be useful to place in another dimension in the future for saleability, e.g. dim_economic_stats
- **yearly_income**
 - stored as text, not numeric, has \$, commas, and k values.
 - Name change to “annual_income”
- **total_debt**
 - stored as text with currency signs, should be numeric
- credit_score
 - all good, all are three digits as should be, and between 300-850, **safe**
- num_credit_cards
 - all numeric, **safe**
- **employment_status**
 - not clean, many different levels for same meaning with typos, spaces, and casing issues, should be standardized into categories:
 - Student, Unemployed, Employed, Retired, Self-Employed
- **education_level**
 - not clean, many different levels for same meaning with typos, abbreviations, casing, and spacing issues, should be standardized into categories:
 - Master Degree, High School, Associate Degree, Doctorate, Bachelor Degree

cards_data.csv

- **id**
 - 61 duplicates, must be removed
- **client_id**
 - **safe**
- **card_brand + card_type**
 - Has several layers with typos, messy text, and nulls, must be standardised to categories:
 - Discover, NA, Amex, Mastercard, Visa
 - Prepaid Debit, NA, Credit, Debit
- **card_number**
 - Issues with format of card number stored. All end in either “.0” or “.5”.
 - Edited the “.0”s and flagged “.5”s as we need to decide how we treat it.
 - Also must check that length is reasonable, up to 19 and at least 13
- **expires**
 - Whilst correct, it is inconsistent, text + integer value, instead we want to have a date stamp, and we assume the date of action is always the first of the month for simplicity
- **cvv**
 - always three digits, anything below we assume needs to be filled e.g. 1 = 001
- **num_issues**
 - No information as to what this column means, it is not atomic, must be dropped
- **credit_limit**
 - Contains comas, full stops, and \$
 - Negatives and text should be NA
- **acct_open**
 - Inconsistent dates, can't be more than today, some month and year, some all, some unavailable
 - We leave NULLS on purpose for date
- **card_on_dark_web**
 - **safe**
- **issuer_bank_name**
 - Several typos and levels, must be standardised to categories:
 - JPMorgan Chase, Discover Bank, Citi, Chase Bank, Wells Fargo, Capital One, U.S. Bank, Truist, Bank of America, PNC Bank, Ally Bank
- **issuer_bank_state**
 - standardise to state code only
- **issuer_bank_type**
 - Several levels, standardise to categories:
 - Regional, Online, National
- **issuer_risk_rating**
 - Several levels, standardise to categories:
 - Medium, low

2 Ingestion

During ingestion the file `ingestion_write.py` creates the database in postgres. The file `ingestion_load.py` loads the four datasets into postgres with no manual column engineering, like setting certain columns as INT or DATE, we leave that for transformation. This is also in the case of scaling and future production, we leave ingestion as raw as possible.

3 Transformation

Here are the details and justification of transformation layer:

transactions.csv

- Pre-processing:
 - Firstly, due to 13 million rows, chunk processing was used, instead of cleaning the entire file and loading, we cleaned a chunk of 100,000 rows and loaded, and did this in a c. 130 iteration loop, to increase the efficiency of the loading. This is useful for scalable design.
 - Once finished we create the schema if it is not already created, and we drop the table, to not duplicate the table every time we run the transformation script. We then use SQL CREATE TABLE to manually create the table.
 - DECIMAL(10,2) was used for all prices, like amount, as we only have 2 decimal places for cents, we assume all price to be USD
 - BOOLEAN was used for all flagged variables
 - VARCHAR was used for text and length varied depending on the context, like address always longer than gender.
- Cleaning:
 - **amount:** removed all dollar signs, and made the data type numeric
 - **is_online + merchant_city:** any “online” values in merchant_city go to the new boolean column is_online. Also refills these values to NA, to keep it true to the context of the city.
 - **merchant_state + merchant_country:** hardcoded all 50 states to check if it is unique, including DC and AA (armed forces) which we assume are valid state locations.
 - Then created merchant_country, if a state is present -> United States, else it keeps the original name which used to be in merchant_state. Nulls replaced with NA

- **zip:** assumed to be only from the US states, therefore removed decimals and replaced nulls with NA
- **is_error_tech + is_error_client + error_text:** This was made to differentiate our side error with client side error. This is a boolean that looks for “Technical glitch” in the original error column, and we assume this is the only error on our side. All else is client error, and error text for more descriptive information, the code also removes technical glitch from the text, so it is only for client error messages
- **is_refund:** created to signify a refund for later curated layers, this we assume are all negative amount’s, hence no negative removal.
- **date_id:** created for a curated layer, this is the date surrogate key.

mcc.csv:

- Pre-processing:
 - We create the schema if it is not already created, and we drop the table, to not duplicate the table every time we run the transformation script. We then use SQL CREATE TABLE to manually create the table.
 - INTEGER used for code
 - VARCHAR(200) used for description
 - Only code and description are kept. Notes and updated_by are dropped as they have no analytical value and too many missing values.
- Cleaning:
 - **code:** Removed quotation marks (“”), “MCC” prefix text and whitespace. Filtered out non-numeric rows, specifically NOTE and COMMENT, that were mixed into the data. And converted to integer for proper joining with the transactions table.
 - **description:** Stripped leading and trailing whitespace, then standardized to Title Case for consistency across all merchant category descriptions.
 - **Duplicates:** Duplicate pairs on the combination of **code** and **description** were removed.

Users_data.csv:

- Pre-processing:
 - We create the schema if it is not already created, and we drop the table to not duplicate the table every time we run the transformation script. We then use SQL CREATE TABLE to manually create the table.

- INTEGER was used for variables such as id, birth_year, birth_month, credit_score, and num_credit_cards
- DECIMAL(10,2) was used for yearly_income and total_debt, assuming monetary values
- DECIMAL(10,6) was used for latitude and longitude to preserve geographic precision
- VARCHAR was used for text and categorical fields such as gender, address, employment_status, and education_level
- Cleaning:
 - **id:**
 - Converted to numeric
 - Invalid ids removed
 - Duplicates removed so that one user corresponds to one row
 - **current_age:**
 - Dropped entirely
 - This column is not stable over time and also showed inconsistency with birth_year
 - **retirement_age:**
 - Dropped entirely
 - Its analytical meaning is unclear and therefore not strong enough to justify inclusion in the transformed table
 - **birth_year:**
 - Kept as a stable date-based customer attribute
 - **birth_month:**
 - Kept as a stable date-based customer attribute
 - **gender:**
 - Standardised to title format, e.g. male -> Male
 - **address:**
 - Leading and trailing spaces removed
 - **latitude + longitude:**
 - Kept as geographic attributes for possible future clustering or regional analysis
 - **per_capita_income:**
 - Removed entirely
 - Although it contains monetary information, it is not sufficiently atomic at the single-user level and is better interpreted as contextual data
 - **yearly_income, renamed to annual_income:**
 - Removed \$, commas, full stops used in formatting, and k notation
 - Converted to numeric
 - **total_debt:**

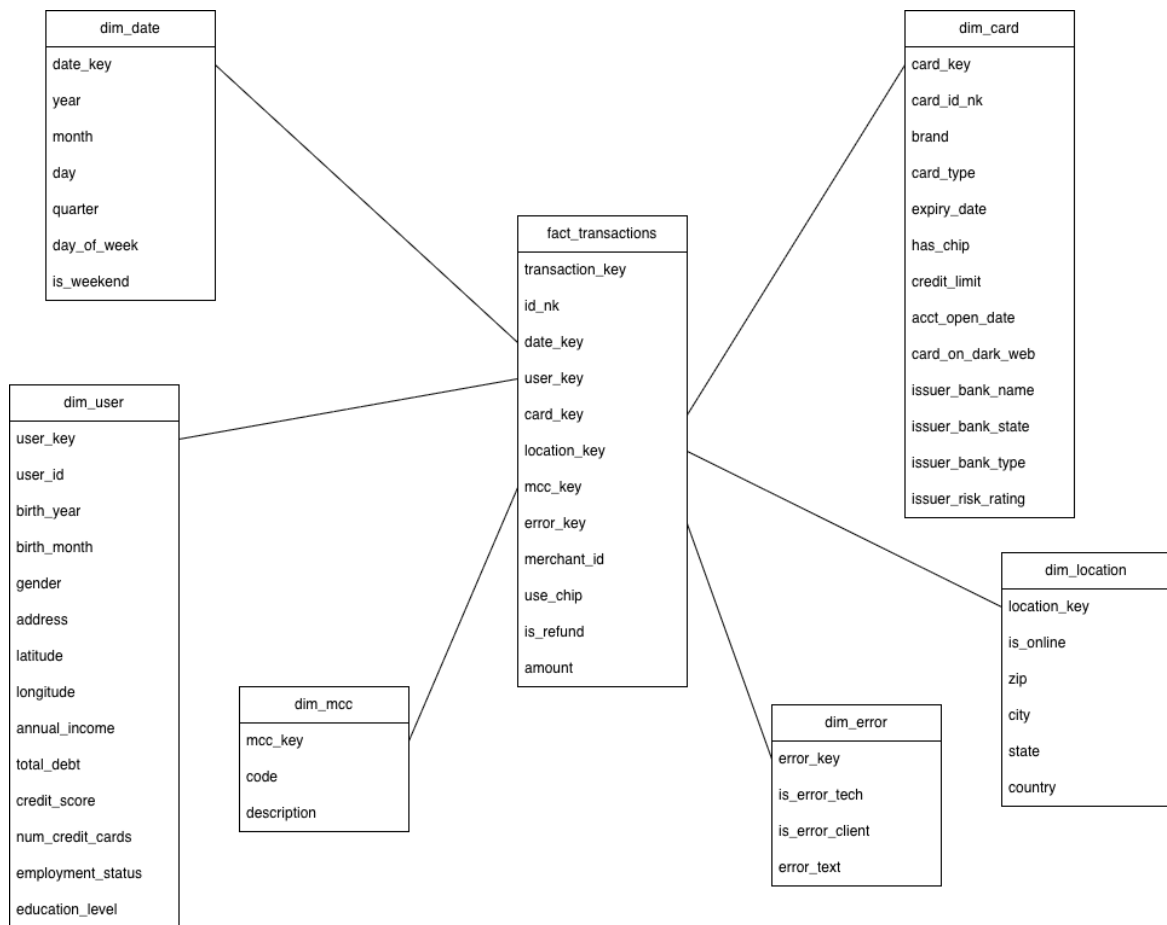
- Cleaned in the same way as yearly_income
- Converted to numeric
- **credit_score:**
 - Converted to numeric
 - Restricted to valid three-digit values only
- **num_credit_cards:**
 - Converted to numeric and kept
- **employment_status:**
 - Standardised using hard-coded mapping logic
 - Final categories: Student, Unemployed, Employed, Retired, Self-Employed
 - Unmatched or missing values set to Unknown
- **education_level:**
 - Standardised using hard-coded mapping logic
 - Final categories: High School, Associate Degree, Bachelor Degree, Master Degree, Doctorate
 - Unmatched or missing values set to Unknown

Cards_data.csv:

- Pre-processing:
 - We create the schema if it is not already created, and we drop the table, to not duplicate the table every time we run the transformation script. We then use SQL CREATE TABLE to manually create the table.
 - DECIMAL(10,2) used for credit_limit, assuming USD currency
 - DATE used for expiry_date and acct_open_date for analytical querying
 - VARCHAR used for categorical fields (e.g. card_brand, issuer_bank_name)
 - INT used for numeric fields such as id and year_pin_last_changed
- Cleaning:
 - **card_brand:**
 - Standardised using regex logic
 - Mapped to categories: Visa, Mastercard, Amex, Discover, NA
 - Handles typos, abbreviations, casing and special characters
 - **card_type:**
 - Standardised using regex logic
 - Mapped to: Credit, Debit, Prepaid Debit, NA
 - Handles inconsistent naming and abbreviations
 - **issuer_bank_name:**

- Standardised using hardcoded mapping dictionary
- Consolidates variations into categories (e.g. JP Morgan Chase, Citi, Wells Fargo)
- Unknown or unmatched values set to NA
- **issuer_bank_state:**
 - Standardised using regex
 - Mapped to: Low, Medium, NA
- **card_number:**
 - Removed trailing .0 or .5 values
 - Enforced valid card number lengths based on brand, i.e. Visa (13, 16 or 19 digits), Mastercard / Discover (16 digits), Amex (15 digits)
- **Duplicates:**
 - Removed duplicates based on combination of **id** and **card_number**.
- **cvv:**
 - Standardised to 3-digit format using zero-padding (e.g. 1 -> 001)
 - Non-numeric or invalid CVVs removed
- **acct_open_date:**
 - Parsed full date formats directly
 - Handled month-year formats (e.g. Feb-99, Feb 1999)
 - Converted to DATE format with assumed day = 1
 - Future dates removed
- **expiry_date**
 - Converted from string format (Jan-24) into DATE
 - Assumed day = 1 for consistency
 - Original column dropped after transformation
- **credit_limit:**
 - Removed currency symbols (\$) and commas
 - Converted “k”notation (e.g. 10k -> 10000)
 - Converted textual values (e.g. “ten thousand”)
 - Invalid or ambiguous values set to NA
 - Negative values assumed to be errors, converted to positive
- **Column removal:**
 - Removed card_number and cvv after validation, due to governance concerns and lack of analytical value

4 Curated



(Figure II: Star-Schema)

Figure II shows the star schema design of the curated layer, with **fact_transactions** at the center and the six surrounding dimension tables.

GRAIN: one row per unique transaction

***This** transaction on **this** date by **this** user with **this** card at **this** location from **this** merchant with **this** mcc*

**all dimensions and the fact table have surrogate keys: "..._key"*

fact_transactions(transaction_key, id_nk, date_key, user_key, card_key, location_key, mcc_key, error_key, merchant_id, use_chip, is_refund, amount)

- transaction_key (surrogate) + id_nk (natural key) to trace back to the source system.
- user_key, card_key, location_key, mcc_key, error_key are foreign keys
- Although we do not have merchant information, merchant_id will be kept for future scalability, in the event we get more merchant information. It also can't be placed in mcc or location dimensions, as a merchant can have many locations and mcc codes. It remains a **degenerate dimension** for now and in the future will be scaled to dim_merchant()
- use_chip is a **degenerate dimension**
- is_refund is a flag for refund calculations, also **degenerate dimension**
- Amount is a fully additive fact across all dimensions, in USD
- ***Note:** *only a 1% sample is loaded from the facts table due to hardware limitations and processing times.*

dim_date(date_key, year, month, day, quarter, day_of_week, is_weekend)

- Useful date analytics for financial purposes derived from transactions data date
- quarter, day_of_week, is_weekend are derived for analytical purposes

dim_error(error_key, is_error_tech, is_error_client, error_client_message)

- error with id = 1 is the default, no error present
- error_tech is our faults, and error_client is client faults
- error_text is the text associated with client error
- This dimension is useful for error rate calculation from either side, and also it is separate to not overpopulate the facts table with text seen in source error

dim_location(location_key, is_online, zip, city, state, country)

- is_online checks if the location is virtual, the rest of the variables are then NA's
- state is only for US and has 52 states (50 + DC + Armed forces)
- zip is only US states, rest NA
- country and city for geographical insights

dim_mcc(mcc_key, code, description)

- code is kept from source as it is an universal standard
- description kept for context of each code
- Not joined with merchant as some merchants have many codes

dim_user(user_key, user_id, birth_year, birth_month, gender, address, latitude, longitude, annual_income, total_debt, credit_score, no_credit_cards, employment_status, education_level)

- user_id needed for dim_user as a natural key for **slowly changing dimension type 2**, as we want to track history of customers, this is not implemented in the current pipeline but is considered for future extension.
- birth_year and birth_month are separate, assuming a date would be severely faulty and misleading, unlike card data where it is more plausible.
- gender kept as is
- We assume address is purely descriptive and street or house number analysis will not be conducted, as we also do not know the specific country of the user, there are many duplicate street names even within one country
- We keep latitude and longitude for scalability in the case that analytics wants to cluster geographical information further.
- annual_income, total_debt, credit_score, no_credit_cards are slightly debatable as they can change a lot, but we assume they will change annually, it is also good to track for credit analysis
- employment_status and education_level are informative for customer analytics

dim_card(card_key, card_id_nk, brand, type, expiry_date, has_chip, credit_limit, acct_open_date, card_on_dark_web, issuer_bank_name, issuer_bank_state, issuer_bank_type, issuer_risk_rating)

- card_id_nk natural key kept in the event of having to use the id publicly (surrogate should always be internal only)
- Keep as **Slowly changing dimension type 1** due to simplicity for OLAP
- brand, type, has_chip, credit_limit kept for analytics
- expiry_date, acct_open_date we assume to be on the first, this is trivial for expiry_date, but for acct_open_date we do it to have easier querying by DATE type. NULLs are allowed to not have text in the column. Useful for active card analysis.
- card_on_dark_web is assumed to be purchases on dark web, we keep for analytical purposes, maybe even fraud if more context given
- Issuer_bank_name, issuer_bank_state, issuer_bank_type, issuer_risk_rating are all analytical insights for the bank-side, useful snowflake design here for future possibility.
- We drop cvv and card_number as it is issue in governance, it also has no real value since we have a unique card id
- pin_last_changed dropped due to no useful analytical purpose, it would also overpopulate the cards dimension if history was preserved. It is also only the year last changed, so anomaly detection is not as strong, if it were date we would keep, e.g. daily changing is suspicious

5 Marts

We made the marts based on departments labelled in the project description: Finance, Customer Analytics, Merchant Partnerships, Fraud, and Leadership.

Each mart is implemented as a set of SQL views built on top of the star schema. The views are saved queries that do not store data but run live against the respective fact and dimension tables every time they are queried. This ensures that the marts are always consistent with the warehouse. For simplicity, marts are implemented within the curated schema, although in production they would be in a separate schema.

5.1 mart_finance

Useful for the Finance department to analyse financial performance and analytics:

view	answers	Key aggregation
_monthly_revenue	Q1: total revenue by month	year, month, SUM(amount)
_refund_rate	Q2: refund %	COUNT/SUM where is_refund
_revenue_by_state	Q3: which states generate most revenue	state, SUM(amount)
_spending_by_category	Q4: highest spending by MCC	mcc code + description, SUM(amount)

This mart provides monthly revenue tracking, refund analysis, revenue breakdowns on a state-level, and spending by merchant category. The revenue views exclude refunds (filtered by **is_refund = FALSE**) to show only positive revenue, while the refund rate view provides both count-based and amount-based refund percentages. Since **is_refund** is boolean, **CASE WHEN** is used to convert it to summable integers (0 and 1). **NULLIF** prevents errors from division by zero, by returning **NULL** instead of crashing when the denominator is zero.

Example query:

```
SELECT *  
FROM curated.mart_finance_monthly_revenue;
```

5.2 mart_customer_analytics

Useful for customer relationship management and general business, views on how customers behave:

view	answers	Key aggregation
_lifetime_value	Q5: LTV per customer	user_key, SUM(amount), first /

		last transaction
_online_vs_instore	Q6: Online vs. in-store behavior	Is_online, SUM(amount), COUNT(*), AVG(amount)
_active_cards	Q7: active cards per customer	user_key, COUNT cards where expiry > now
_suspicious_patterns	Q8: suspicious patterns	user_key, error rates, refund rates

Lifetime value is calculated per user as total spending with the dates of the users respective first and last transaction. The online vs in-store view aggregates by the **is_online** flag from **dim_location**. Active cards are counted per user, by filtering **dim_card** on **expiry_date** against the latest transaction date in the dataset, since **CURRENT_DATE** would mark all cards as expired, given that all transactions occurred before the current date. This ensures only non-expired cards (relative to the dataset's latest transaction date, not today) are included. Suspicious patterns combine client error rates, refund rates and technical error counts, per user, to find potential anomalies.

Example query:

```
SELECT *
FROM curated.mart_customer_analytics_online_vs_instore;
```

5.3 mart_merchant_partnerships

The Merchant mart is useful for merchant relationship management and analysis of the industry.

view	answers	Key aggregation
_volume	Q9: highest transaction volume	merchant_id, COUNT
_industry_growth	Q10: fastest growing industries	mcc, year + month, COUNT trend (monthly...)
_error_rates	Q11: highest error rates	merchant_id, COUNT(error), COUNT(*), error_rate
_geographic_revenue	Q12: revenue by geography	state + country, SUM(amount)

The transaction volume is aggregated directly from the fact table by **merchant_id** as it is a degenerate dimension (scalable in the future). Industry growth provides a monthly breakdown by MCC code which enables analysis on trends in MCC over time. Error rates count any transaction with either a technical or client error, per merchant. Lastly, geographic revenue groups the revenue by country and state from **dim_location**, with **NULL** states representing online / international transactions.

Example query:

```
SELECT *
FROM curated.mart_merchant_partnerships_volume LIMIT 10;
```

5.4 Fraud

The fraud mart supports anomaly detection and fraud investigation.

view	answers	Key aggregation
_high_error_users	Compare to Q11: Users with high client-side error rates	user_key, error_count / total_count
_refund_anomalies	Compare to Q2: User-card combinations with high refund rates	user_key, card_key, refund_count / total_count

The view **_high_error_users** shows users with disproportionately high client-side (i.e. not technical) error rates which may indicate misuse or that the respective accounts are compromised. **_refund_anomalies** further shows the refund rate on a user-card level, i.e. for different users, each with a certain amount of cards, since a high refund rate on a specific card is more suspicious than a high overall refund rate for a user, as this may actually indicate cases of card theft for instance.

**For future scalability, it would be interesting to have a more specific date of pin last changed, as then anomaly analysis can be done with the variable. Also more context on card_on_dark_web needed.*

Example query:

```
SELECT *
FROM curated.mart_fraud_high_error_users
WHERE total_transactions > 10
LIMIT 10;
```

5.5 Leadership

Provides an executive dashboard layer.

view	answers	Key aggregation
_kpi_overview	<i>"Finance cannot confidently report monthly revenue" + "Leadership wants dashboards"</i>	SUM(amount), COUNT(*), COUNT(DISTINCT user_key), AVG(amount)
_monthly_trends	Q1 but as growth trends	year, month, SUM(amount), COUNT(*), month-over-month % change
_top_states	Q3	state, SUM(amount),

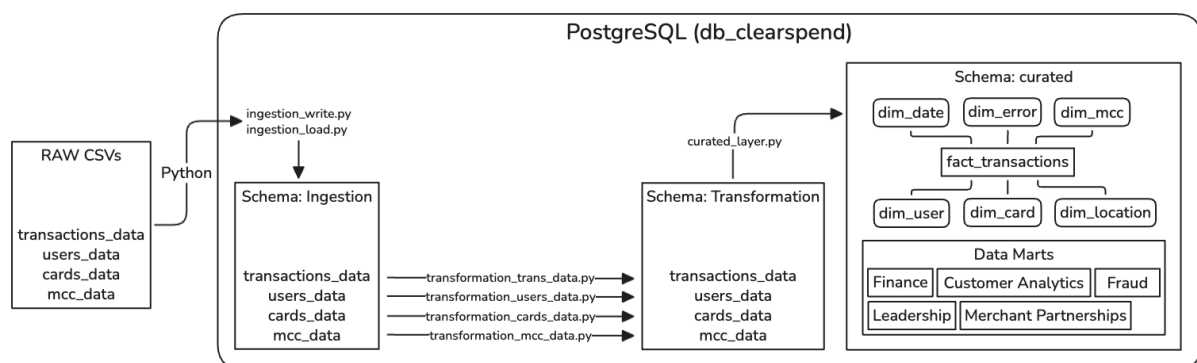
		COUNT(*), RANK()
_online_vs_instore	Q6	is_online, SUM(amount), COUNT(*), AVG(amount)

The KPI overview is a single-row summary of the total revenue, transaction count, unique customer count and average transaction amount. Monthly trends first groups transactions by month, and then computes the month-over-month percentage changes in revenue and transaction volume using **LAG()**. Top states uses **RANK()** to rank states by revenue, and the online vs instore view mirrors the customer analytics version but placed within the leadership mart along other KPI views, avoiding the need to switch marts for central decision making.

Example query:

```
SELECT *
FROM curated.mart_leadership_kpi_overview;
```

6 Conclusion



(Figure III: Data Flow Diagram)

The pipeline runs end-to-end from raw CSV ingestion, through transformation, to a fully queryable star schema with five business-aligned data mart views for the respective department's analytical needs. The result is a reproducible warehouse that can be rebuilt by rerunning the pipeline scripts, as well as be extended with new dimensions and marts as the business grows.

Figure III illustrates the full pipeline from the source files to the curated layer, including the data marts detailed above, while also showing which scripts are responsible for each stage.